

Práctica 1

Entorno de desarrollo Android: SDK, primer proyecto y dispositivos virtuales

Configuración del entorno, caracterización de dispositivos, compatibilidad y pruebas en el emulador

Nombre y apellidos: Daniel Núñez Uribe.....

Fecha de entrega:

Resultados de aprendizaje y criterios de evaluación

CE	Criterio de evaluación trabajado (RA1)	Evaluación
c	Se han instalado, configurado y utilizado entornos de trabajo para el desarrollo de aplicaciones para dispositivos móviles.	Se evalúa en esta práctica
h	Se han utilizado emuladores para comprobar el funcionamiento de las aplicaciones.	Se evalúa en esta práctica
d	Se han identificado configuraciones que clasifican los dispositivos móviles en base a sus características.	Se trabaja; se evalúa en el cuestionario de la unidad
e	Se han descrito perfiles que establecen la relación entre el dispositivo y la aplicación.	Se trabaja; se evalúa en el cuestionario de la unidad
f, g	Se ha analizado la estructura de aplicaciones existentes... / Se han realizado modificaciones sobre aplicaciones existentes.	Se introducen; se evalúan en la UT2

1. Directrices generales de entrega

Estas indicaciones son de aplicación general a esta práctica, salvo que en algún apartado de este guion se indique expresamente lo contrario. Léelas con atención antes de comenzar, porque forman parte de la evaluación.

Formato del documento

- El documento se entrega en letra Arial, tamaño 12, salvo que se indique otra cosa para un elemento concreto.
- El texto debe estar bien estructurado, con títulos, subtítulos, párrafos y, cuando sea necesario, listas o tablas.
- Se cuidarán especialmente la presentación, la ortografía y la redacción.
- Las capturas de pantalla y las imágenes deben tener un tamaño suficiente para poder leerse con claridad.

- No se dejarán grandes espacios en blanco ni páginas prácticamente vacías.
- Si se indica una extensión orientativa, debe respetarse: no se debe alcanzar artificialmente aumentando el tamaño de letra, los márgenes o los espacios.

Nombre del archivo

Se entregan dos archivos, nombrados siguiendo este formato:

Formato del nombre de los archivos

Documento: **nombre_apellido_pmdm_ut1_p1.pdf** Por ejemplo:

carlos_luque_pmdm_ut1_p1.pdf

Proyecto comprimido: **nombre_apellido_pmdm_ut1_p1.zip** Por ejemplo:

carlos_luque_pmdm_ut1_p1.zip

El nombre debe escribirse en minúsculas, sin tildes ni caracteres especiales.

Identificación, autoría y entrega

- El documento debe incluir nombre y apellidos, módulo, unidad de trabajo y fecha de entrega, datos que ya aparecen recogidos en la portada de este guion.
- El trabajo se realiza de forma individual, salvo que el profesorado indique lo contrario.
- Se considera copia tanto la reproducción total como la parcial del trabajo de otro alumno o alumna. Si se detecta copia entre dos o más trabajos, la actividad podrá considerarse no superada para todas las personas implicadas.
- El uso de fuentes externas está permitido salvo que se indique lo contrario, pero debes comprender y saber explicar todo el contenido que entregues.
- En esta práctica no está autorizado el uso de herramientas de inteligencia artificial (incluido el asistente Gemini de Android Studio). Cualquier contenido generado con ellas se considerará una fuente externa no autorizada.
- El trabajo se entrega por el medio y dentro del plazo que indique el profesorado. Un borrador que no se haya enviado no se considera entregado.

Antes de entregar

Comprueba lo siguiente antes de enviar tu trabajo:

- Que has completado todos los apartados y tablas solicitados en este guion.
- Que el documento tiene el formato indicado (Arial 12, apartados, tablas).
- Que los archivos tienen el nombre correcto.
- Que no hay errores ortográficos ni de redacción.
- Que se han incluido todas las capturas solicitadas, en el orden del guion, y que se ven con claridad.
- Que el proyecto comprimido se abre y compila, y que es la versión correcta y definitiva.

2. Objetivos

- Revisar y configurar el SDK de Android con el SDK Manager e identificar sus paquetes y herramientas.
- Crear un proyecto Android con Jetpack Compose y analizar su configuración de compilación (niveles de API y versiones de las herramientas).
- Crear dispositivos virtuales con características distintas y clasificarlos por su pantalla, su versión de Android y su arquitectura.
- Ejecutar una aplicación en varios dispositivos virtuales y verificar con datos reales la caracterización realizada.
- Comprobar cómo el minSdk de la aplicación decide en qué dispositivos se puede instalar.
- Simular condiciones reales de uso con los controles extendidos del emulador y utilizar herramientas del SDK desde la terminal.
- Realizar modificaciones controladas sobre la aplicación y observar su efecto.

3. Materiales y requisitos previos

- Android Studio instalado y operativo (ya disponible en el aula).
- Virtualización por hardware activada en el equipo (ya configurada en el aula).
- Unos 10 GB libres en disco para las imágenes de sistema y los dispositivos virtuales.
- Haber leído el complemento de apuntes de la UT1 (clasificación de dispositivos, compatibilidad y SDK): lo necesitarás para completar las tablas.
- No es necesario tener ningún dispositivo virtual creado de antemano: se crearán durante la práctica.

4. Desarrollo

La práctica se organiza en ocho partes. Cada parte se apoya en la anterior, así que realízalas en orden. Pega las capturas en tu documento en el mismo orden en que se piden, cada una con un pequeño título que indique a qué paso corresponde.

Parte	Contenido
1	Revisar y configurar el SDK
2	Crear el proyecto y analizar su configuración de compilación
3	Explorar la estructura y el código del proyecto
4	Crear y caracterizar los dispositivos virtuales
5	Ejecutar la aplicación y verificar la caracterización
6	Comprobar la compatibilidad entre el dispositivo y la aplicación
7	Simular condiciones reales con el emulador y usar el SDK desde la terminal
8	Modificar la aplicación

Parte 1 — Revisar y configurar el SDK

1. Abre Android Studio. Desde la pantalla de bienvenida pulsa More Actions → SDK Manager (o, con un proyecto abierto, Tools → SDK Manager).
2. Anota la ruta que aparece en Android SDK Location. La necesitarás en la Parte 7.
C:\Users\ldrunn\AppData\Local\Android\Sdk
3. En la pestaña **SDK Platforms**, activa la casilla Show Package Details (abajo a la derecha). Así verás, dentro de cada versión de Android, su plataforma y sus imágenes de sistema.
4. Comprueba que tienes instalados, como mínimo, los siguientes paquetes. Si falta alguno, márcalo y pulsa Apply (la descarga puede tardar varios minutos):
 - La **plataforma** de la versión de Android más reciente disponible y una **imagen de sistema x86_64 con Google Play** de esa misma versión.
 - Una **imagen de sistema x86_64** (con Google Play o con Google APIs) de una versión **dos niveles de API por debajo** de la anterior.
5. En la pestaña **SDK Tools**, activa también Show Package Details y localiza la versión instalada de Android SDK Build-Tools, Android SDK Platform-Tools, Android Emulator y Android SDK Command-line Tools. Si las Command-line Tools no están instaladas, instala la versión latest.

Captura de pantalla: pestaña SDK Platforms con Show Package Details activado, donde se vean las imágenes de sistema instaladas.

Android 17.0 ("CinnamonBun")				
☒ Android SDK Platform 37.2	37.2	1	Installed	
☒ Sources for Android 37.2	37.2	1	Installed	
☒ 16 KB Page Size Google Play Intel x86_64 Atom System Image	37.2	5	Installed	
☐ 16 KB Page Size Google APIs Intel x86_64 Atom System Image	37.2	5	Not installed	
Android 15.0 ("VanillaIceCream")				
☐ Android SDK Platform 35	35	2	Not installed	
☐ Sources for Android 35	35	1	Not installed	
☐ AOSP ATD Intel x86_64 Atom System Image	35	1	Not installed	
☐ Intel x86_64 Atom System Image	35	2	Not installed	
☐ Google APIs Intel x86_64 Atom System Image	35	9	Not installed	
☐ Google Play Intel x86_64 Atom System Image	35	9	Not installed	
☒ 16 KB Page Size Google Play Intel x86_64 Atom System Image	35	5	Installed	

Captura de pantalla: pestaña SDK Tools con las versiones de las herramientas.

Android SDK Build-Tools	Android SDK Command-line Tools (latest)
☒ 37.0.0	☒ Android SDK Command-line Tools (latest)
☒ Android Emulator	37.1.11
☐ Android Emulator hypervisor driver (installer)	2.2.0
☐ Android Project Templates	0.1.1
☒ Android SDK Platform-Tools	37.0.1

Tabla 1 — Estado del SDK. Complétala con los datos que acabas de consultar:

Elemento	Valor observado
Ubicación del SDK	C:\Users\drunn\AppData\Local\Android\Sdk
Plataformas instaladas (niveles de API)	Android SDK Platform 37.2
Imágenes de sistema instaladas (nivel de API, ABI y variante)	API 37.2 – Intel x86_64, Google Play API 35.0 – Intel x86_64, Google Play
Versión de SDK Build-Tools	37.0
Versión de SDK Platform-Tools	37.0.1
Versión de Android Emulator	37.1.11
Versión de Android Studio (Help → About)	Android Studio Quail 4 2026.1.4 Patch 1

Parte 2 — Crear el proyecto y analizar su configuración de compilación

1. En la pantalla de bienvenida selecciona New Project.
2. En el catálogo de plantillas, elige **Empty Activity** (categoría Phone and Tablet). Esta plantilla ya viene configurada con Jetpack Compose.
3. Rellena los datos del proyecto:
 - Name: **MiPrimeraApp**

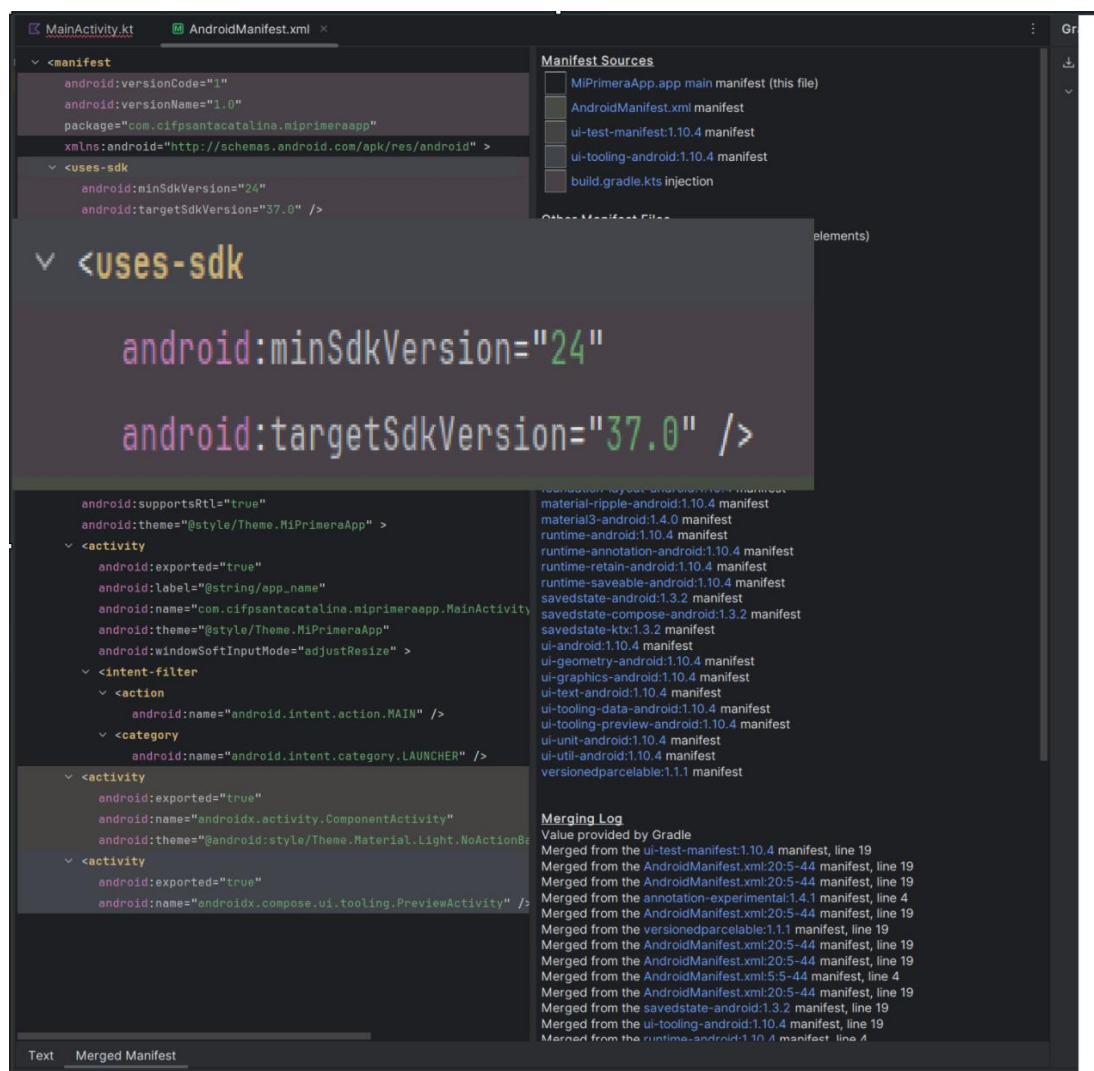
- Package name: el que indique el profesor (por ejemplo, com.cifpsantacatalina.miprimeraapp)
 - Minimum SDK: el que recomiende el propio asistente. Fíjate en el porcentaje de dispositivos que indica que podrán ejecutar la aplicación.
 - Build configuration language: Kotlin DSL (build.gradle.kts), si el asistente lo pregunta.
4. Pulsa Finish y espera a que Gradle sincronice el proyecto. La primera sincronización puede tardar varios minutos: es un proceso normal.
 5. Cambia la vista del panel lateral izquierdo al modo **Project** (desplegable de la parte superior del panel) para ver todos los ficheros. Abre `app/build.gradle.kts`, `gradle/libs.versions.toml` y `gradle/wrapper/gradle-wrapper.properties`.

Tabla 2 — Configuración de compilación. Localiza cada parámetro y complétala:

Parámetro	Valor	Fichero donde aparece
applicationId	<code>com.cifpsantacatalina.miprimeraapp</code>	build.gradle.kts
namespace	<code>com.cifpsantacatalina.miprimeraapp</code>	build.gradle.kts
minSdk	<code>minSdk = 24</code>	build.gradle.kts
targetSdk	<code>targetSdk = 37</code>	build.gradle.kts
compileSdk	<code>compileSdk { version = release(version = 37) }</code>	build.gradle.kts
versionCode y versionName	<code>versionCode = 1 versionName = "1.0"</code>	build.gradle.kts
Versión de Gradle	<code>/gradle-9.6.0-bin.zip</code>	gradle-wrapper.properties
Versión del Android Gradle Plugin (agp)	<code>agp = "9.4.1"</code>	libs.version.toml
Versión de Kotlin	<code>kotlin = "2.2.10"</code>	libs.version.toml
Versión de la BOM de Compose (composeBom)	<code>composeBom = "2026.02.01"</code>	libs.version.toml

6. Vuelve a la vista **Android** del panel lateral. Abre `manifests/AndroidManifest.xml` y, en la parte inferior del editor, pulsa la pestaña **Merged Manifest**.
7. Localiza en el manifiesto fusionado la etiqueta `<uses-sdk>` y comprueba que sus atributos `minSdkVersion` y `targetSdkVersion` coinciden con los valores de la Tabla 2, aunque tú no los hayas escrito en el `AndroidManifest.xml`.

Captura de pantalla: pestaña Merged Manifest con la etiqueta uses-sdk visible.



Parte 3 — Explorar la estructura y el código del proyecto

Con la vista **Android** del panel lateral, localiza la carpeta manifests, la carpeta kotlin+java (o java) con el fichero MainActivity.kt, la carpeta res con sus subcarpetas y el grupo Gradle Scripts. Después abre MainActivity.kt y localiza la clase que hereda de ComponentActivity, el método onCreate(), la llamada a setContentView {}, la función Greeting marcada con @Composable, la función GreetingPreview marcada con @Preview y el tema MiPrimeraAppTheme.

Tabla 3 — Análisis de la estructura. Complétala con lo observado:

Elemento	Ubicación en el proyecto	Para qué sirve
Activity principal	App/kotlin+java/nombrePaquete/ MainActivity.kt	Supongo que será la primera vista que verá el usuario al acceder a la app.
Método onCreate()	Dentro de las activities, en mi caso y de momento, solo dentro de MainActivity.kt	Es un método que se invocará cuando la activity se genera en la app, imagino que inicializa sus variables.
Llamada setContentView { }	Dentro de las activities, en mi caso y de momento, solo dentro de MainActivity.kt	Es la función principal de Jetpack Compose, sirve para construir el contenido visual de la activity.
Función @Composable Greeting	Dentro de las activities, en mi caso y de momento, solo dentro de MainActivity.kt	Es una primera función para mostrar datos por pantalla, como el "Hello World!" en otros lenguajes.
Función @Preview	Dentro de las activities, en mi caso y de momento, solo dentro de MainActivity.kt	Función que compone una especie de UI de demo sin necesidad de un dispositivo emulado.
Tema MiPrimeraAppTheme	Dentro de la llamada setContentView { }	Tema de colores general para la activity y todos sus elementos hijos
AndroidManifest.xml	App\src\main	Archivo de config esencial para la app, sirve para comunicarle al SO y a Google Play los requisitos de la app, por ejemplo.
Carpeta res/	App\src\main	Almacenar los recursos de la aplicación, imagen, audio...
build.gradle.kts (Module :app)	App\	Definición de compilación del proyecto.

Parte 4 — Crear y caracterizar los dispositivos virtuales

Un dispositivo virtual (AVD) combina **un perfil de hardware** y **una imagen de sistema**. Vas a crear dos, con características claramente distintas, para poder compararlos.

1. Abre el **Device Manager** (View → Tool Windows → Device Manager, o su icono en la barra lateral derecha) y pulsa + → Create Virtual Device.
2. Categoría **Phone**: selecciona un perfil de gama alta actual (por ejemplo, Pixel 8 o posterior). Antes de pasar a la siguiente pantalla, anota la resolución y la densidad que aparecen junto al perfil. Como imagen de sistema, elige la versión más reciente con Google Play y ABI x86_64. Nombra el dispositivo **AVD_Telefono** y finaliza.
3. Categoría **Tablet**: selecciona un perfil de unas 10 pulgadas (por ejemplo, Pixel Tablet o Medium Tablet) y anota también su resolución y densidad. Como imagen de sistema, elige la de dos niveles de API por debajo que preparaste en la Parte 1. Nombra el dispositivo **AVD_Tablet** y finaliza.

Tabla 4 — Datos de los dispositivos virtuales:

AVD	Perfil de hardware	Resolución (px)	Densidad (dpi)	Nivel de API	ABI
AVD_Telefono	Pixel 8 Pro	1344x2992	480	34+	x86_64
AVD_Tablet	Pixel Tablet	2560x1600	320	33+	x86_64

A partir de esos datos, calcula la clasificación de cada dispositivo aplicando lo visto en el complemento de apuntes. Para el ancho en vertical usa el lado corto de la resolución y, para el ancho en horizontal, el lado largo.

Tabla 5 — Clasificación calculada:

AVD	Factor de densidad	Ancho en vertical (dp)	Ancho en horizontal (dp)	Grupo de densidad	Clase de ancho (vertical / horizontal)
AVD_Telefono	3	448	997	xxhdpi	Compact / Expanded
AVD_Tablet	2	800	1280	xhdpi	Medium/Large

Debajo de la tabla, escribe las operaciones que has hecho para calcular los anchos en dp de uno de los dos dispositivos.

Para **móvil**, teniendo en cuenta que: $dp = px \times 160 / dpi$ y que el factor de densidad = $dpi / 160$;

Factor de densidad (fd): **$480 / 160 = 3$** ,

Ancho en vertical (**dp**) = $px / fd = 1344 / 3 = 448 \text{ dp}$, Ancho en horizontal (**dp**) = $px / fd = 2560 / 3 = 997 \text{ dp}$.

Parte 5 — Ejecutar la aplicación y verificar la caracterización

Ahora vas a comprobar tus cálculos con los datos reales que el propio dispositivo virtual comunica a la aplicación. Para ello añadirás a la aplicación una función que muestra en pantalla la información del dispositivo.

Antes de empezar

En esta unidad todavía no se ha estudiado en profundidad la sintaxis de Kotlin ni de Jetpack Compose (se hará en las unidades siguientes). El código que se pide añadir se proporciona ya escrito: el objetivo es que sepas localizar dónde va, copiarlo en el lugar correcto y observar su efecto, no que lo escribas desde cero.

1. Abre MainActivity.kt y copia la siguiente función **al final del fichero**, fuera de cualquier otra función:

MainActivity.kt — nueva función

```
@Composable
fun InfoDispositivo(modifier: Modifier = Modifier) {
    val configuracion = LocalConfiguration.current
    Column(modifier = modifier.padding(16.dp)) {
        Text(text = "Ancho de la ventana: ${configuracion.screenWidthDp} dp")
        Text(text = "Alto de la ventana: ${configuracion.screenHeightDp} dp")
        Text(text = "Densidad: ${configuracion.densityDpi} dpi")
        Text(text = "Nivel de API: ${Build.VERSION.SDK_INT}")
        Text(text = "ABI principal: ${Build.SUPPORTED_ABIS.first()}")
    }
}
```

2. Dentro de onCreate(), localiza el bloque Scaffold que llama a Greeting(...) y sustituye esa llamada a Greeting por este bloque, para que se muestren el saludo y la información del dispositivo, uno debajo del otro:

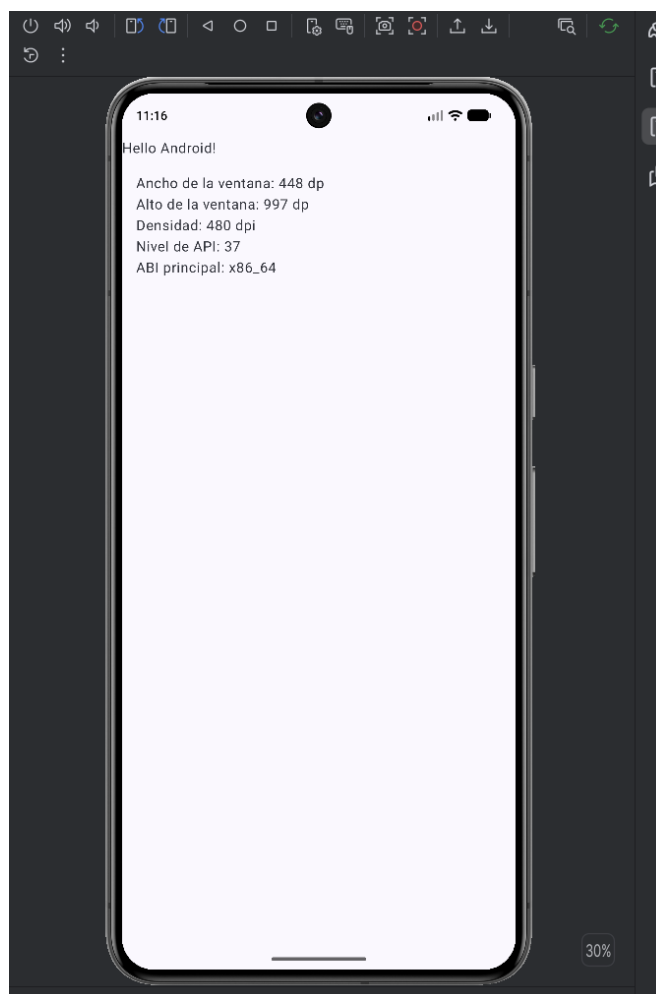
MainActivity.kt — contenido de setContent

```
Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
    Column(modifier = Modifier.padding(innerPadding)) {
        Greeting(name = "Android")
        InfoDispositivo()
    }
}
```

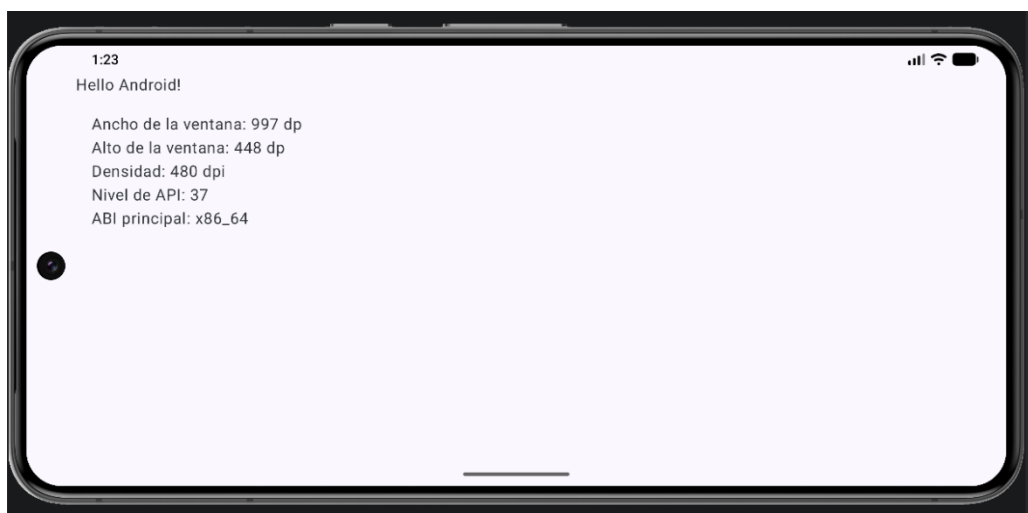
Si Android Studio marca en rojo Column, LocalConfiguration, Build o dp, sitúa el cursor sobre el nombre y pulsa **Alt + Enter** para añadir automáticamente el import que falte. Si aparece algún aviso en amarillo, la aplicación compila igualmente.

3. Selecciona **AVD_Telefono** en el desplegable de dispositivos (junto al botón Run) y pulsa Run. Espera a que el emulador arranque y se instale la aplicación.
4. Con la aplicación en marcha, gira el emulador con los botones de rotación de su barra de herramientas y observa cómo cambian los valores de ancho y alto.
5. Repite los dos pasos anteriores con **AVD_Tablet**.

Captura de pantalla: la aplicación ejecutándose en AVD_Telefono en vertical.



Captura de pantalla: la aplicación ejecutándose en AVD_Telefono en horizontal.



Captura de pantalla: la aplicación ejecutándose en AVD_Tablet en vertical y en horizontal (dos capturas).



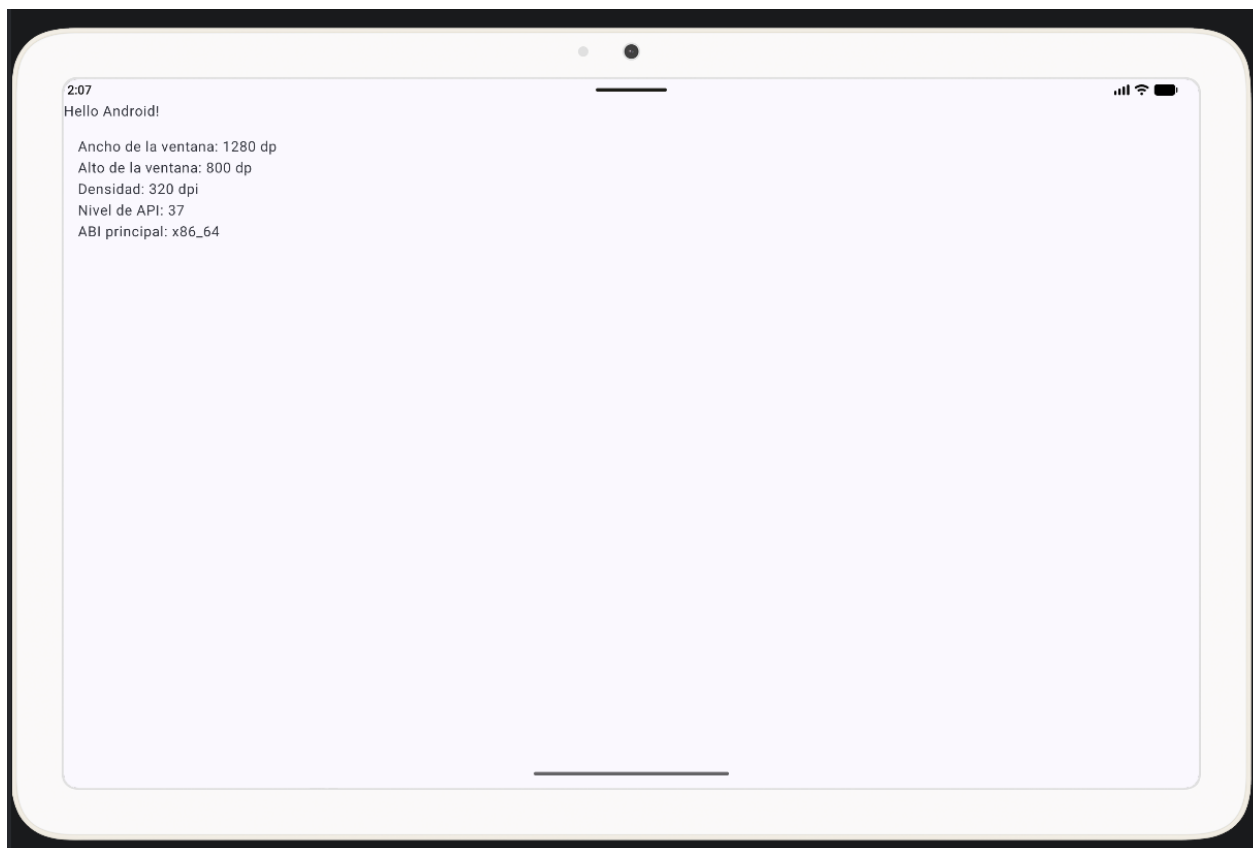


Tabla 6 — Verificación. Compara los valores medidos con los calculados en la Tabla 5:

AVD	Ancho medido vertical / horizontal (dp)	Densidad medida (dpi)	Nivel de API medido	ABI medida	¿Coincide con tu cálculo?
AVD_Telefono	448 / 997	480	37	x86_64	Sí ☒
AVD_Tablet	800 / 1280	320	37	x86_64	Sí ☒

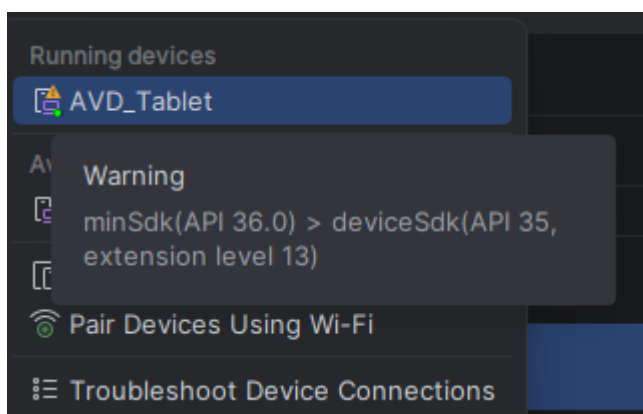
Puede haber diferencias de muy pocos dp debidas al redondeo. Si la diferencia es grande, revisa tu cálculo antes de seguir. Debajo de la tabla, responde brevemente: ¿por qué es útil, antes de publicar una aplicación, probarla en configuraciones tan distintas como las de esta práctica? Relaciónalo con el concepto de fragmentación.

Respuesta: **El mercado móvil es amplísimo en cuanto a características de hardware de los equipos sobre los que correrá nuestra aplicación, esto es lo que se conoce como fragmentación. Es importante probar en tantas configuraciones como se pueda para asegurarnos de que nuestra aplicación es eficiente y ágil en todos los equipos posibles con sus diferentes capacidades, así como que la interfaz se adecúa (Es responsive) al tamaño, formato y disposición de cada dispositivo.**

Parte 6 — Comprobar la compatibilidad entre el dispositivo y la aplicación

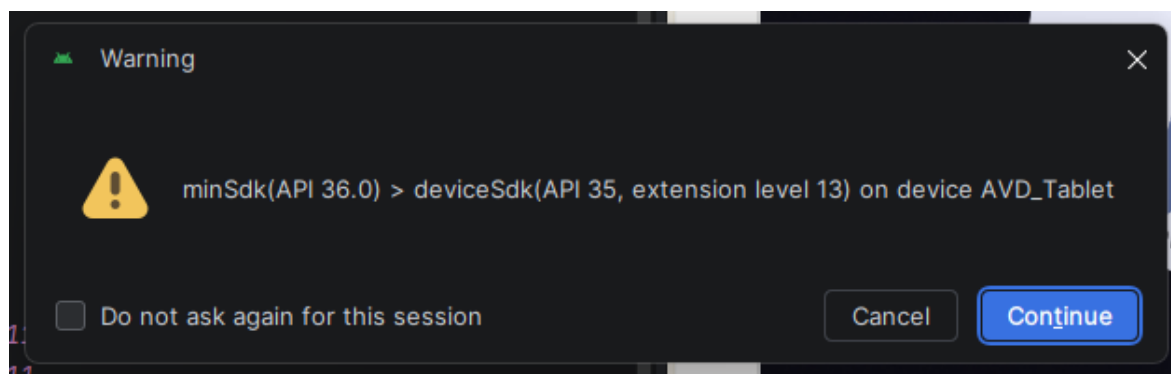
En esta parte vas a provocar a propósito una incompatibilidad para ver cómo el minSdk decide en qué dispositivos se puede instalar una aplicación.

1. Consulta en la Tabla 4 el nivel de API de AVD_Tablet.
2. Abre `app/build.gradle.kts` y cambia el valor de **minSdk** por uno **mayor que el nivel de API de AVD_Tablet**, sin superar el valor de `compileSdk`. Por ejemplo, si AVD_Tablet tiene nivel de API 35, escribe `minSdk = 36`.
3. Pulsa **Sync Now** en la barra amarilla que aparece en la parte superior del editor.(o el símbolo del elefante con la fecha)
4. Selecciona AVD_Tablet y pulsa Run. Lee con atención el mensaje que muestra Android Studio (en el selector de dispositivos, en un cuadro de diálogo o en la ventana Run).



5. Selecciona AVD_Telefono y pulsa Run. Comprueba que en este dispositivo la aplicación sí se instala.
6. Abre de nuevo la pestaña **Merged Manifest** y comprueba el nuevo valor de `minSdkVersion`.
7. Restaura el valor original de `minSdk` y vuelve a sincronizar con Sync Now.

Captura de pantalla: el mensaje de error o aviso que aparece al intentar ejecutar la aplicación en AVD_Tablet.



Captura de pantalla: la pestaña Merged Manifest con el minSdkVersion modificado.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.cifpsantacatalina.miprimeraapp"
    android:versionCode="1"
    android:versionName="1.0">

    <uses-sdk
        android:minSdkVersion="36.0"
        android:targetSdkVersion="37.0" />

    <application
        android:allowBackup="true"
        android:appComponentFactory="androidx.core.app.CoreComponentFactory"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.MiPrimeraApp" >
        <activity
            android:exported="true"
            android:label="@string/app_name"
            android:name="com.cifpsantacatalina.miprimeraapp.MainActivity"
            android:theme="@style/Theme.MiPrimeraApp"
            android:windowSoftInputMode="adjustResize" >
            <intent-filter>
                <action
                    android:name="android.intent.action.MAIN" />
                <category
                    android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:exported="true"
            android:name="androidx.activity.ComponentActivity"
            android:theme="@android:style/Theme.Material.Light.NoActionBar" />
        <activity
            android:exported="true"
            android:name="androidx.compose.ui.tooling.PreviewActivity" />
    </application>
</manifest>
```

Debajo de las capturas, copia el texto exacto del mensaje que ha mostrado Android Studio y explica en dos o tres líneas por qué la aplicación se instala en un dispositivo y no en el otro.

Respuesta:

minSdk(API 36.0) > deviceSdk(API 35, extensión level 13) on device AVD_Tablet

Indica que la API mínima para la instalación de la app se ha establecido en 36.0 y que la del dispositivo AVD_Tablet es la 35. Como mi AVD_Phone tiene la API 37, que es superior, contiene las funcionalidades de la 36, y permite instalarla sin problema.

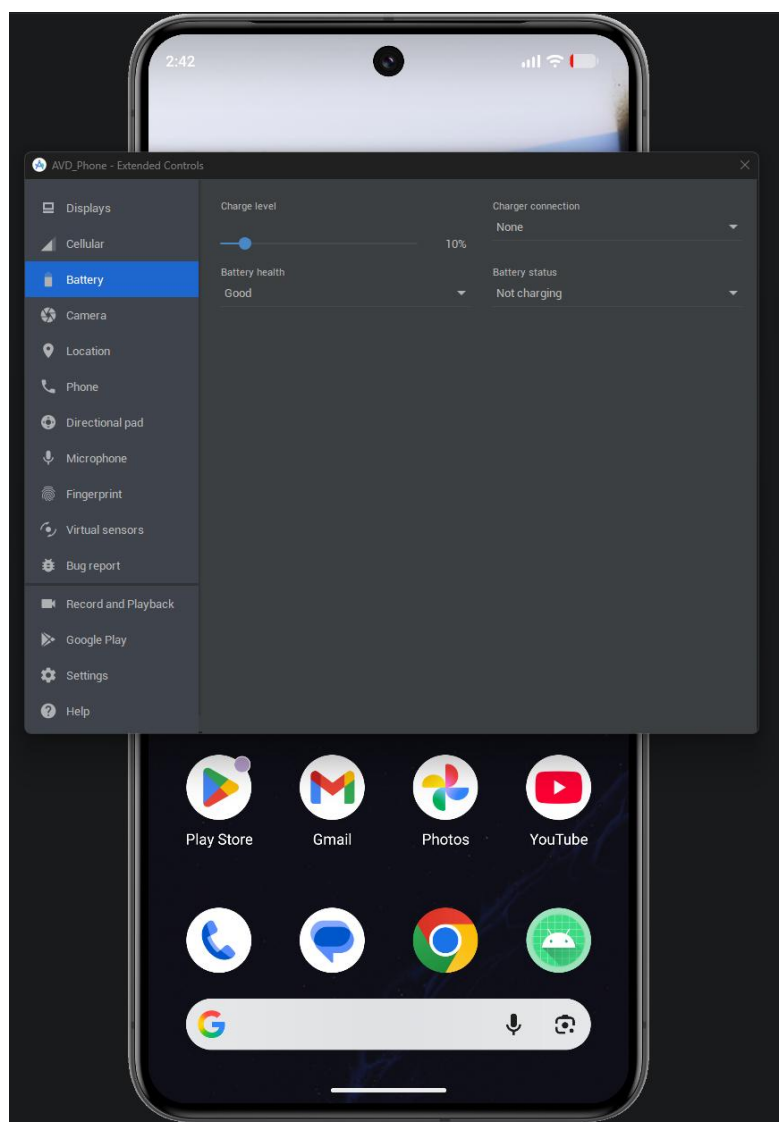
Parte 7 — Simular condiciones reales con el emulador y usar el SDK desde la terminal

El emulador permite simular situaciones que en un dispositivo físico serían difíciles de reproducir. Todas las opciones de esta parte están en los **controles extendidos** del emulador: botón de los tres puntos (: o ...) de su barra de herramientas, o Extended Controls si el emulador se muestra dentro de Android Studio. Trabaja con AVD_Telefono.

7.1. Batería

1. En Extended Controls → Battery, cambia Charger connection a None y baja Charge level al 10 %.
2. Observa el icono de batería de la barra de estado y las notificaciones que muestre el sistema.

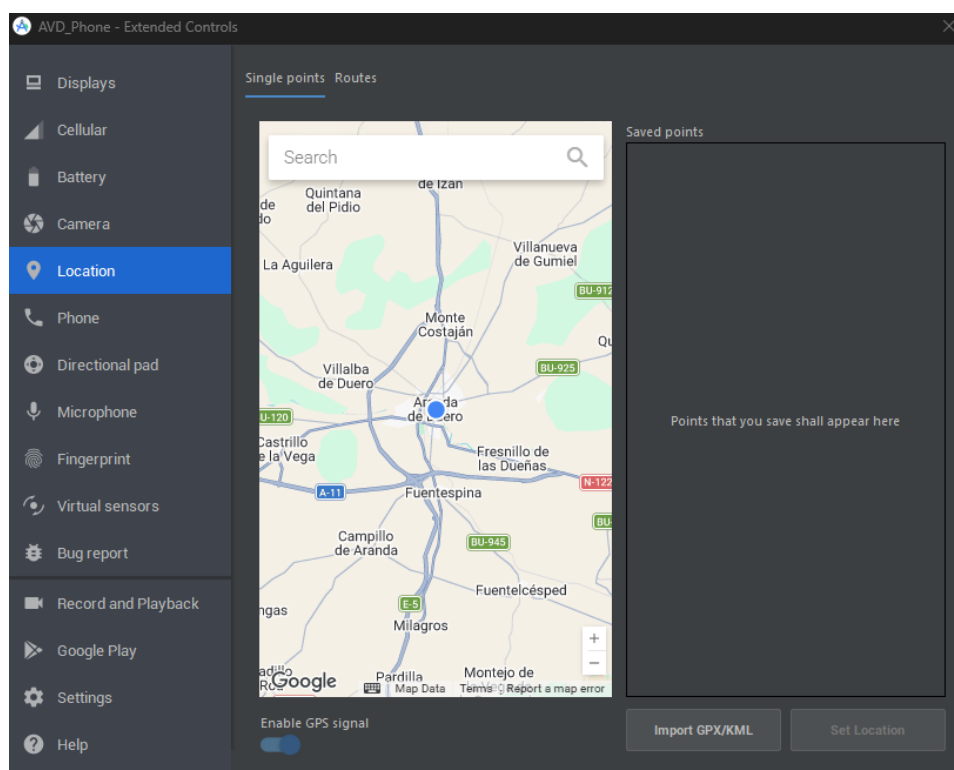
Captura de pantalla: el emulador con la batería al 10 % y sin cargador.



7.2. Ubicación

1. En Extended Controls → Location, busca «Aranda de Duero» (o introduce las coordenadas 41.6704, -3.6892) y pulsa Set Location.
2. Abre la aplicación Google Maps del emulador y pulsa el botón de ubicación actual. Acepta los permisos que te pida.

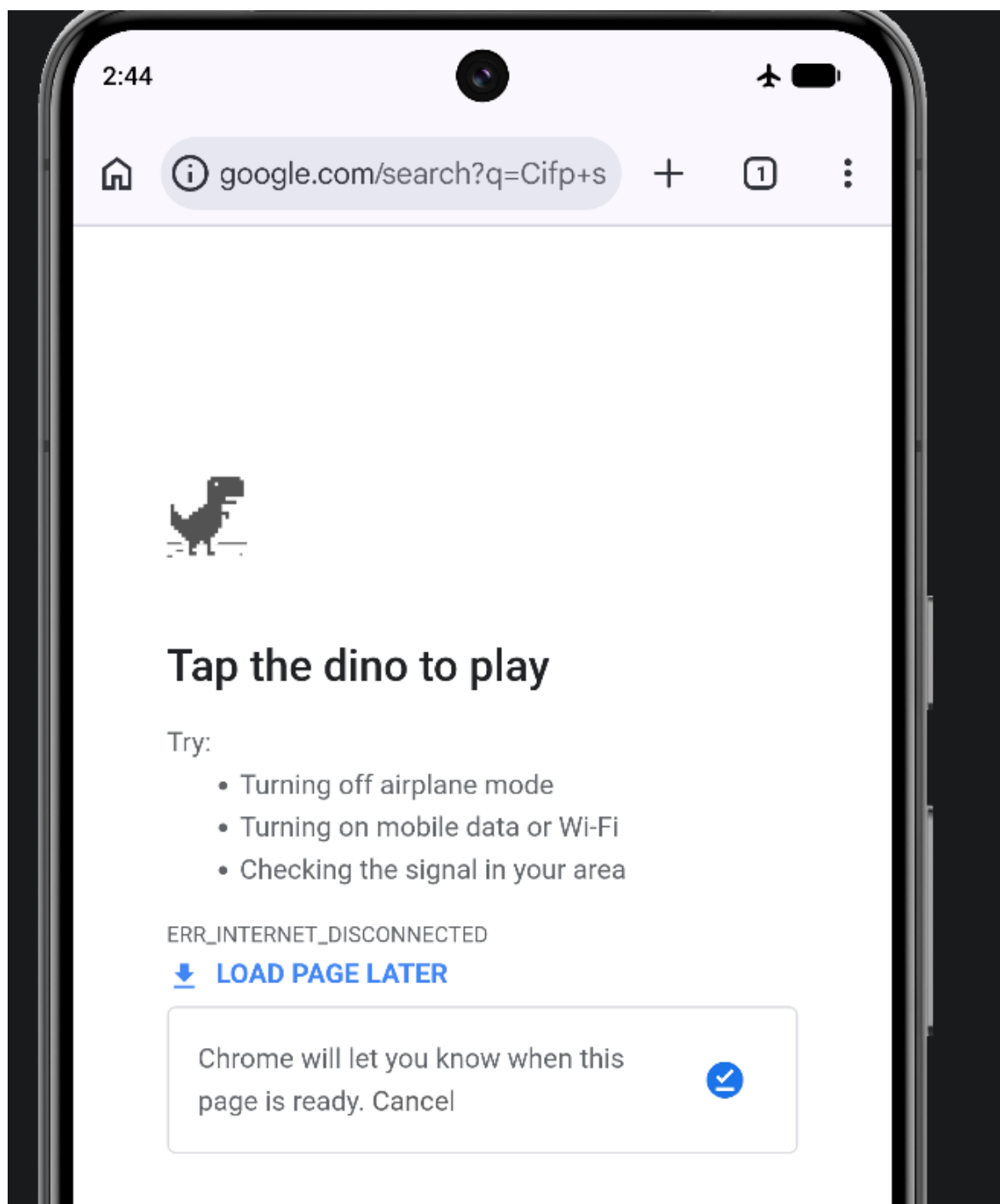
Captura de pantalla: Google Maps mostrando la ubicación simulada.



7.3. Conectividad

1. Desliza hacia abajo desde la parte superior de la pantalla del emulador y activa el **modo avión**.
2. Abre el navegador Chrome del emulador e intenta cargar cualquier página. Observa el resultado.
3. Desactiva el modo avión.

Captura de pantalla: Chrome sin conexión con el modo avión activado.



7.4. Arranque del emulador

1. Cierra el emulador. Por defecto, al cerrarlo se guarda su estado para arrancar después más deprisa (Quick Boot).
2. En el Device Manager, abre el menú de AVD_Telefono (:) y elige **Cold Boot**. Mide aproximadamente cuánto tarda en arrancar. **Tiempo de inicio: 29s**
3. Ciérralo de nuevo y arráncalo normalmente con el botón de inicio. Mide el tiempo otra vez. **Tiempo de inicio: 4s**
4. Anota los dos tiempos en tu documento.

7.5. Herramientas del SDK desde la terminal

Con AVD_Telefono arrancado (y solo ese emulador en marcha), abre la ventana **Terminal** de Android Studio (View → Tool Windows → Terminal). En Windows, la terminal por defecto es PowerShell. Ejecuta estas órdenes una a una. Si en la Parte 1 anotaste una ubicación del SDK distinta de la habitual, sustituye la ruta por la tuya.

Terminal (PowerShell)

```
cd "$env:LOCALAPPDATA\Android\Sdk\platform-tools"  
.\adb devices  
.\adb shell getprop ro.build.version.sdk  
cd "$env:LOCALAPPDATA\Android\Sdk\emulator"  
.\emulator -list-avds
```

Captura de pantalla: la terminal con el resultado de las cuatro órdenes.

```
PS C:\Users\drunn\AppData\Local\Android\Sdk\platform-tools> .\adb devices  
List of devices attached  
emulator-5554    device  
  
PS C:\Users\drunn\AppData\Local\Android\Sdk\platform-tools> .\adb shell getprop ro.build.version.sdk  
37  
  
PS C:\Users\drunn\AppData\Local\Android\Sdk\platform-tools> cd "$env:C:\Users\drunn\AppData\Local\Android\Sdk\emulator"  
  
PS C:\Users\drunn\AppData\Local\Android\Sdk\emulator> .\emulator -list-avds  
AVD_Phone  
AVD_Tablet  
  
PS C:\Users\drunn\AppData\Local\Android\Sdk\emulator> █
```

Devices devuelve el número de dispositivos emulados activos, “Shell getprop...” entiendo que devuelve la versión de compilación de la app del proyecto abierto, y “**.emulator -list-avds**” lista los nombres de los dispositivos emulados cargados en el proyecto.

Debajo de la captura, indica qué información devuelve cada orden y comprueba que el nivel de API que devuelve adb coincide con el de la Tabla 4.

Parte 8 — Modificar la aplicación

En esta última parte vas a hacer tres cambios sobre la aplicación y a observar su efecto. Continúa ejecutando la aplicación en AVD_Telefono.

8.1. Cambiar el texto

1. En MainActivity.kt, localiza la llamada Greeting(name = "Android") que añadiste en la Parte 5 y cambia "Android" por tu nombre.
2. Ejecuta de nuevo la aplicación y comprueba que aparece el nuevo saludo.

Captura de pantalla: la aplicación mostrando el saludo con tu nombre.

8.2. Añadir un botón con contador

Sustituye la función Greeting completa por el siguiente código:

MainActivity.kt — función Greeting

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    var contador by remember { mutableStateOf(0) }
    Column(modifier = modifier) {
        Text(text = "Hola, $name!")
        Button(onClick = { contador++ }) {
            Text(text = "Pulsado $contador veces")
        }
    }
}
```

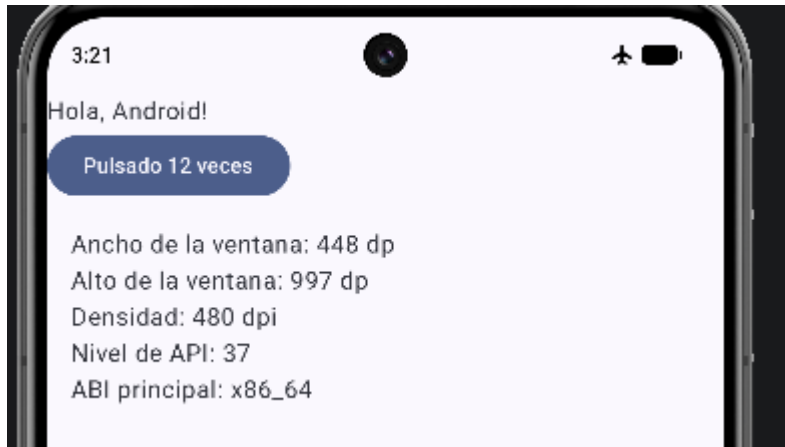
Si Android Studio marca en rojo Button, remember, mutableStateOf o by, sitúa el cursor sobre el nombre y pulsa **Alt + Enter** para añadir el import que falte.

Qué hace este código, a grandes rasgos

remember { mutableStateOf(0) } crea una variable contador cuyo valor la interfaz «recuerda» mientras se redibuja; Column coloca el texto y el botón uno debajo del otro; y onClick indica qué ocurre al pulsar el botón. Todo esto se estudiará en detalle en unidades posteriores.

1. Ejecuta la aplicación y pulsa el botón varias veces. Comprueba que el contador aumenta.
2. Con el contador en un número mayor que 0, **gira el emulador**. Observa qué ocurre con el contador y anótalo.
3. Cambia remember por **rememberSaveable** (añade el import con Alt + Enter), vuelve a ejecutar, pulsa el botón varias veces y gira de nuevo el emulador. Anota qué ha cambiado.

Captura de pantalla: la aplicación con el botón, tras pulsarlo varias veces (el contador debe ser mayor que 0).



Captura de pantalla: la aplicación girada con rememberSaveable, conservando el valor del contador.



La activity se regenera con onCreate() cada vez que se rota así que el valor del contador de la propia activity, se perderá y se reseteará a 0.

Debajo de las capturas, explica en dos o tres líneas por qué se pierde el valor del contador al girar el dispositivo con remember. Pista: recuerda qué le ocurre a la Activity cuando cambia la configuración del dispositivo.

8.3. Cambiar el color del tema

1. Abre el fichero `ui/theme/Theme.kt` y localiza el parámetro `dynamicColor: Boolean = true` de la función `MiPrimeraAppTheme`. Cámbialo a **false**. Con el color dinámico activado, en Android 12 o superior la aplicación toma los colores del fondo de pantalla del dispositivo e ignora los definidos en el proyecto.
2. Abre `ui/theme/Color.kt` y cambia el valor de **Purple40** (el color principal del tema claro) por otro a tu elección, por ejemplo `Color(0xFFC1440E)`. Si tu emulador usa el tema oscuro, cambia también **Purple80**.
3. Ejecuta la aplicación y comprueba que el botón aparece con el nuevo color.

Captura de pantalla: la aplicación con el botón en el nuevo color.

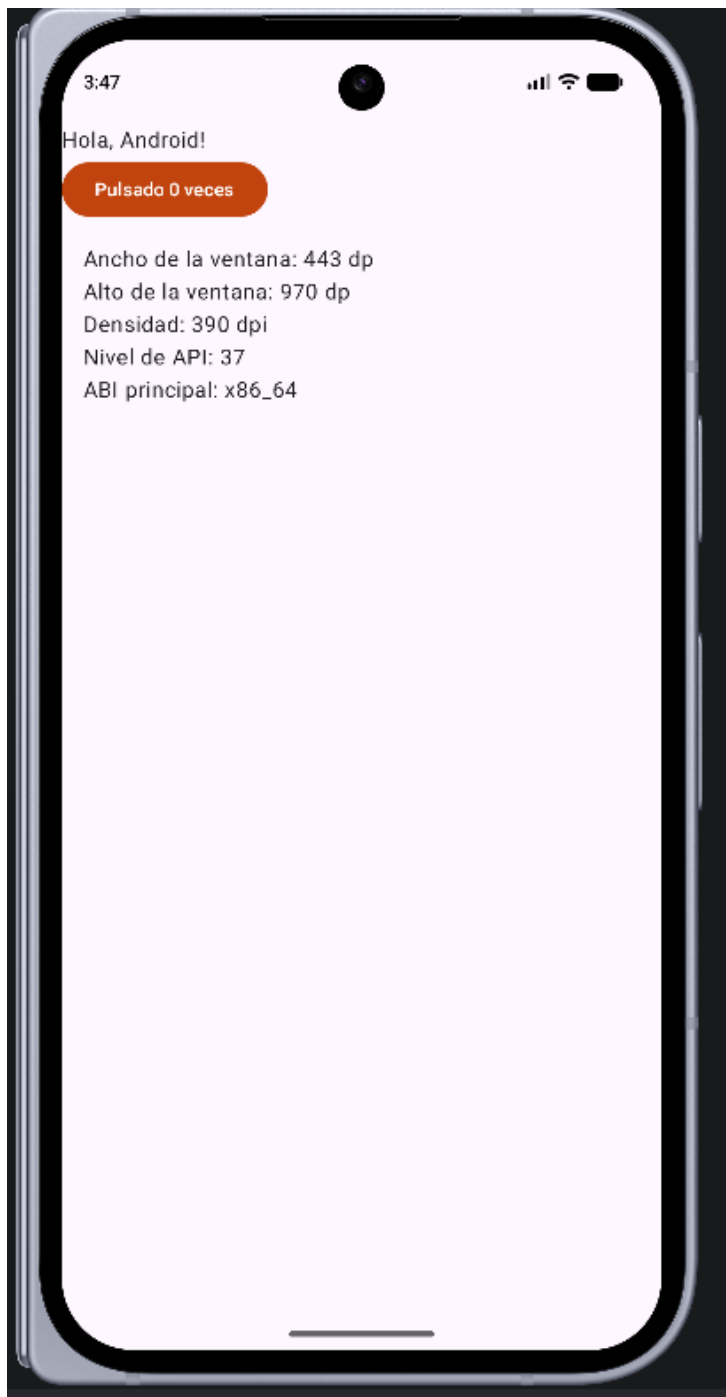


Reto de ampliación (opcional)

Crea un tercer dispositivo virtual de tipo plegable (categoría Phone, perfil con «Fold» en el nombre). Ejecuta la aplicación, pliega y despliega el dispositivo desde la barra de herramientas del emulador y comprueba cómo cambian el ancho y la clase de tamaño de ventana. Documenta el resultado con capturas y añade una fila para este dispositivo en las Tablas 4, 5 y 6.

Ampliación – Capturas:

Fold, Cerrado y vertical



Fold, abierto y vertical:



Tabla 4

AVD	Perfil de hardware	Resolución (px)	Densidad (dpi)	Nivel de API	ABI
AVD_Foldable (Closed)	Pixel 10 Pro	2364x1080	390	36+	x86_64
AVD_Foldable (Open)	Pixel 10 Pro	2152x2076	390	36+	x86_64

Tabla 5

AVD	Factor de densidad	Ancho en vertical (dp)	Ancho en horizontal (dp)	Grupo de densidad	Clase de ancho (vertical / horizontal)
AVD_Foldable (Closed)	2,4375	443	970	xhdpi	Compact/ Expanded
AVD_Foldable (Open)	2,4375	852	883	xhdpi	Expanded/ Expanded

Tabla 6

AVD	Ancho medido vertical / horizontal (dp)	Densidad medida (dpi)	Nivel de API medido	ABI medida	¿Coincide con tu cálculo?
AVD_Foldable (Closed)	443 / 970	390	37	x86_64	Sí ☒
AVD_Foldable (Open)	852 / 883	320	37	x86_64	Sí ☒

5. Entregable

Se entregan los dos archivos indicados en las directrices generales:

- **Documento PDF** con las Tablas 1 a 6 completas, las operaciones de cálculo de la Parte 4, todas las capturas solicitadas en el orden del guion y las respuestas breves que se piden en las Partes 5, 6, 7 y 8.
- **Proyecto MiPrimeraApp comprimido** en formato ZIP, con todas las modificaciones de la Parte 8 aplicadas y el minSdk restaurado a su valor original. Puedes generarlo desde Android Studio con File → Export → Export to Zip File.

6. Rúbrica de evaluación

Cada ítem se valora con uno de los tres niveles. La puntuación del nivel intermedio se sitúa en torno al 60 % de la puntuación máxima del ítem y la del nivel bajo, entre el 0 % y el 30 %.

Ítem	Nivel alto	Nivel medio	Nivel bajo	Pts.
1. Configuración del SDK (CE c)	El SDK está revisado y completo; la Tabla 1 recoge con precisión ubicación, plataformas, imágenes (API, ABI y variante) y versiones de las herramientas.	La Tabla 1 está completa, pero con algún dato impreciso o sin detallar las imágenes de sistema.	Faltan paquetes necesarios, la tabla está incompleta o no hay capturas.	1,5
2. Proyecto y configuración de compilación (CE c)	El proyecto se crea correctamente; las Tablas 2 y 3 identifican bien cada parámetro, versión y elemento, con su fichero; se aporta el manifiesto fusionado.	Las tablas están completas con uno o dos errores menores, o falta la captura del manifiesto fusionado.	Tablas incompletas o con errores importantes en los niveles de API o en la estructura.	1,5
3. Compatibilidad dispositivo-aplicación (CE c)	Se provoca y documenta la incompatibilidad por minSdk con el mensaje exacto, se verifica el manifiesto fusionado y la explicación es correcta. El minSdk queda restaurado.	Se provoca la incompatibilidad, pero la explicación es incompleta o falta alguna evidencia.	No se realiza la prueba o no se documenta.	1
4. Modificaciones de la aplicación (CE c)	Los tres cambios (texto, contador con rememberSaveable y color) funcionan, y se explica correctamente la pérdida del estado al girar.	Funcionan dos de los tres cambios, o la explicación del giro es incompleta.	Funciona como mucho un cambio o la aplicación no compila.	1
5. Dispositivos virtuales y clasificación (CE h)	Se crean dos AVD con perfil e imagen claramente distintos; las Tablas 4 y 5 son correctas, con las operaciones de cálculo.	Los AVD son correctos, pero hay errores en algún cálculo o clasificación.	Un solo AVD, AVD sin diferencias relevantes o tablas sin completar.	2
6. Pruebas en el emulador (CE h)	La aplicación se ejecuta en ambos AVD y orientaciones; la Tabla 6 contrasta bien medido y calculado; se documentan batería, ubicación, conectividad, arranque y órdenes de terminal.	Se ejecuta en ambos AVD, pero falta la verificación completa o una o dos simulaciones de la Parte 7.	Solo se ejecuta en un AVD o faltan la mayoría de las pruebas.	2
7. Documentación y presentación	El documento sigue las directrices, está ordenado según el guion, las capturas se leen bien y tienen título; el ZIP se abre y compila.	Pequeños fallos de formato, orden o nitidez de las capturas.	Documento desordenado, capturas ilegibles o nombre de archivos incorrecto.	1
TOTAL				10